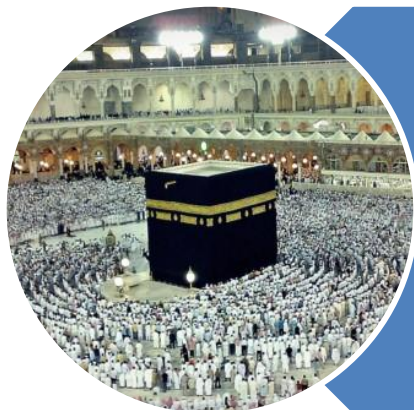


رَبِّ اشْرَحْ لِي صَدْرِي وَيَسِّرْ لِي أَمْرِي وَاحْلُلْ عُقْدَةً مِنْ لِسَانِي يَفْقَهُوا قَوْلِي



O my Lord! Expand for me my chest [with assurance] and ease for me my task and untie the knot from my tongue that they may understand my speech. **(Quran 20 : 25-28)**

Linked List

Introduction

C

P

C

S

2

0

4

3

Linked List

- Abstract Data Type (ADT) or Data Structure
 - A linked list is a sequence of nodes in which each **node** is **linked to the node following it**.



C

P

C

S

2

0

4

4

Array vs **Linked List**

- | | |
|---|---|
| <ul style="list-style-type: none">• Static | <ul style="list-style-type: none">• Dynamic |
| <ul style="list-style-type: none">• Contiguous | <ul style="list-style-type: none">• Non - Contiguous |
| <ul style="list-style-type: none">• Direct Access | <ul style="list-style-type: none">• No Direct Access |
| <ul style="list-style-type: none">• Homogenous | <ul style="list-style-type: none">• Homogenous |
| <ul style="list-style-type: none">• Linear | <ul style="list-style-type: none">• Linear |
| <ul style="list-style-type: none">• Physical | <ul style="list-style-type: none">• Physical |

C

P

C

S

2

0

4

5

Array vs **Linked List**

- Advantages
 - Multiple values
 - Direct Access

- Disadvantages
 - Static
 - Slow Insertion (shifting)
 - Slow Deletion (shifting)

- Advantages
 - **Dynamic**
 - **Fast Insertion**
 - **Fast Deletion**
- Disadvantage
 - **No Direct Access**
 - **Accessing is slow**

Linked List



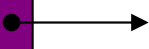
- A linked list is an ordered collection of data
 - Each element (generally called **nodes**) contains the location of the next element in the list
- Each **node** essentially has two parts:
 - The data part
 - If this was a list of student records, for example, the data here may consist of a name, PID, social security number, address, phone, email, etc.
 - The link part
 - This link is used to chain the nodes together.
 - It simply contains a reference that points to the next node in the linked list
 - Variable is often called “next”



Node – Class Declaration

data

next



```
public class LNode {
    private int data;
    private LNode next;

    // Constructor 1
    public LNode(int i) {
        this(i, null);
    }

    // Constructor 2
    public LNode(int i, LNode n) {
        data = i;
        next = n;
    }
}
```

Node - Structure



- Not limited to **TWO** parts only i.e. data & next
- It could be **20+ separate fields** of information storing name, address, phone, email, etc.



```

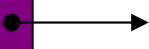
public class LNode {
    private String name;
    private String id;
    private int gradePoints;
    private LNode next;

    // more code here
};
    
```


Node – Class Declaration

data

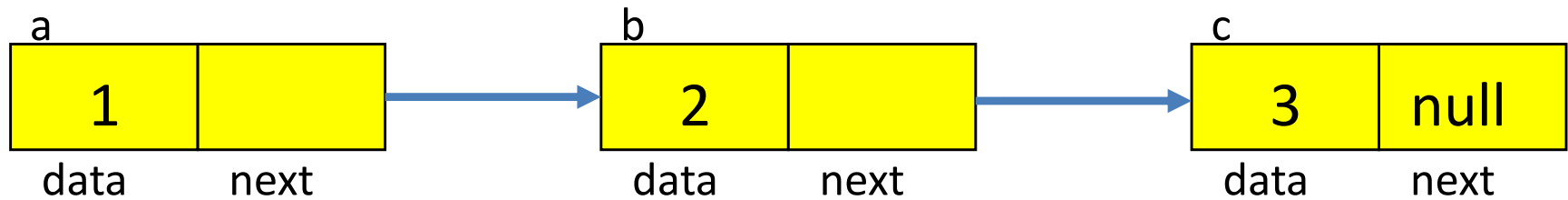
next



```
public class LNode {  
    private int data;  
    private LNode next;  
  
    // Constructor 1  
    public LNode(int i) {  
        this(i, null);  
    }  
  
    // Constructor 2  
    public LNode(int i, LNode n) {  
        data = i;  
        next = n;  
    }  
}
```

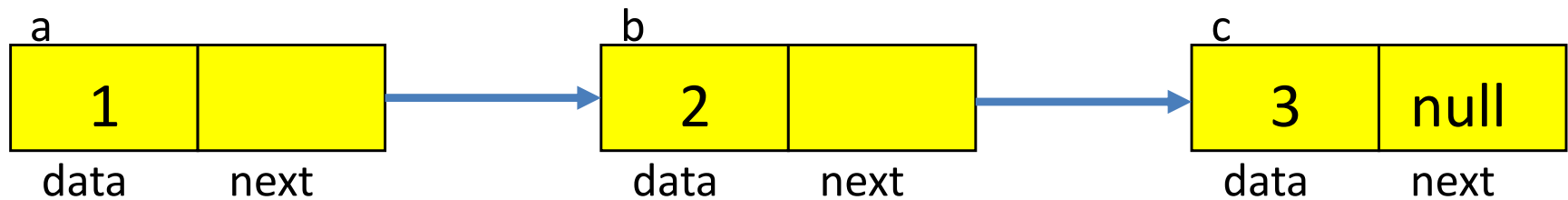
Node - Creation

```
LLnode a = new LLnode(1);
LLnode b = new LLnode(2);
LLnode c = new LLnode(3);
```



```
a.next = b;
b.next = c;
```

Node - Access



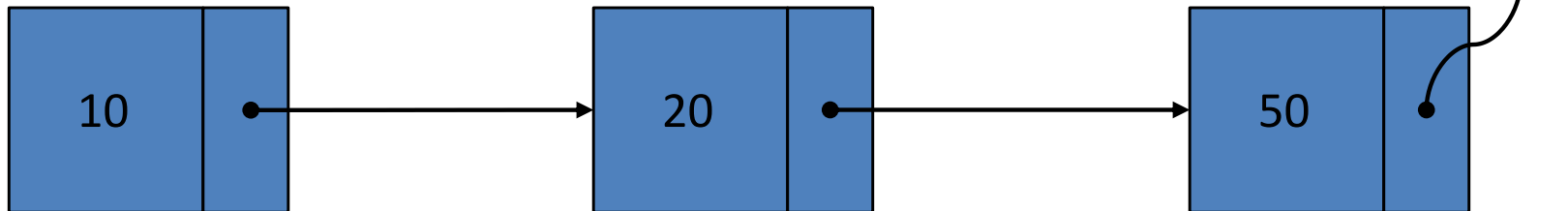
```

System.out.print(a.data) ;
System.out.print(a.next.data) ;
System.out.print(a.next.next.data) ;
System.out.print(b.next.data) ;
System.out.print(c.next.data) ;
  
```

1 is printed
2 is printed
3 is printed
3 is printed
ERROR!

Linked List

- You can think of each node as a **record**
 - The **first part** of the record is all the necessary **data**
 - The final part of the record is a field that stores a pointer to the **next node** in the list



- So who knows where is node 50?
 - The next of node 20.
- And who knows where is node 20?
 - The next of node 10.

C

P

C

S

2

0

4

Linked List

- Head of the list

- Each node of the list is created dynamically and points to the next node
 - So from the first node, we can get to the second node
 - From the second node, we can get to the third node
 - and so on
- You must have a reference variable that simply points to the first node of the list
 - Usually this variable is named **head**
 - This variable has ONE goal in life: **point to the first node!**

“No Bonus” marks for this course anymore

C

P

C

S

2

0

4

Linked List

- So we will need two Java Classes
 1. The first class is for the individual nodes of the linked list
`class Llnode`
 - We use this class to make individual node objects.
 2. The second class is a class that **controls the list** and stores access to the actual list
`class LinkedList`
 - This class defines the head/front reference variable
 - It can also define a “tail” (or end of list) reference variable
 - Depends on your own implementation
 - This class also defines many methods that operate on the list
 - insertion, deletion, searching, printing nodes, etc.

C

P

C

S

2

0

4

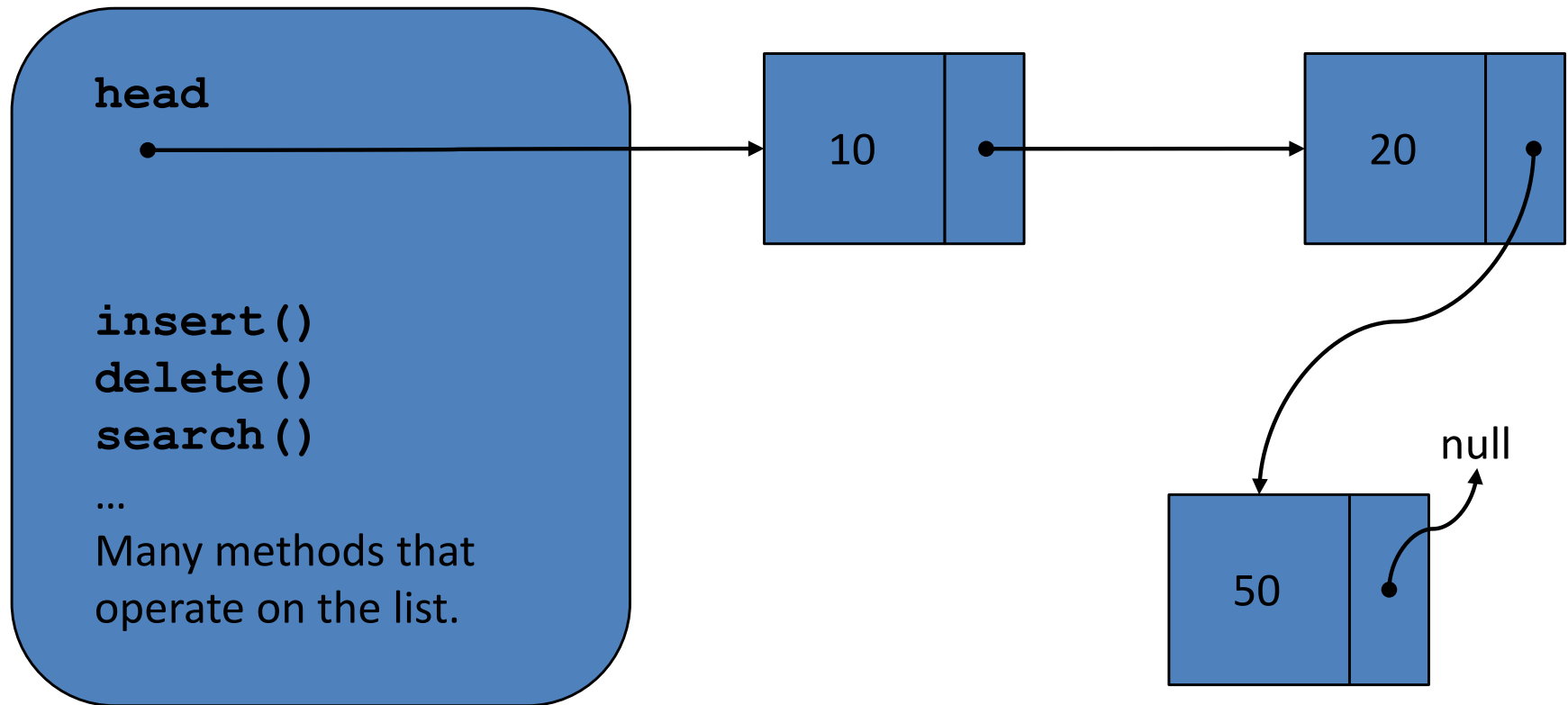
Class – LinkedList

```
public class LinkedList {  
    private LLnode head;  
  
    // Constructor  
    public LinkedList() {  
        head = null;  
    }
```

```
    public boolean isEmpty() {  
        if (head == null)  
            return true;  
        else  
            return false;  
    }
```

```
    // Example Method to check if list is empty  
    public boolean isEmpty() {  
        return head == null;  
    }  
  
    // All methods will be written here  
}
```

Visual Representation- **LinkedList** class



Linked List

Operations

C

P

C

S

2

0

4

Operations

- Is Empty
- Traversing
- Searching
- Insertion
- Deletion
- Merging

C

P

C

S

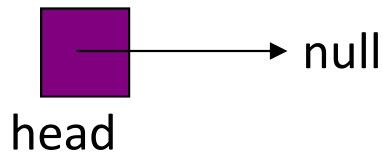
2

0

4

Linked List – **isEmpty()**

- There is no nodes in the list
- Head points to null



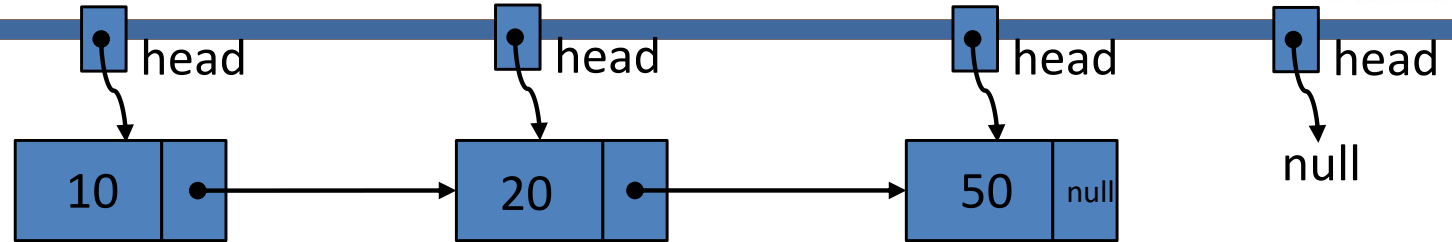
An empty linked list

- isFull()
 - Not Needed

```
public class LinkedList {
    private LLnode head;
    // Constructor
    public LinkedList() {
        head = null;
    }

    public boolean isEmpty() {
        return head == null;
    }
}
```

"No Bonus" marks for this course anymore



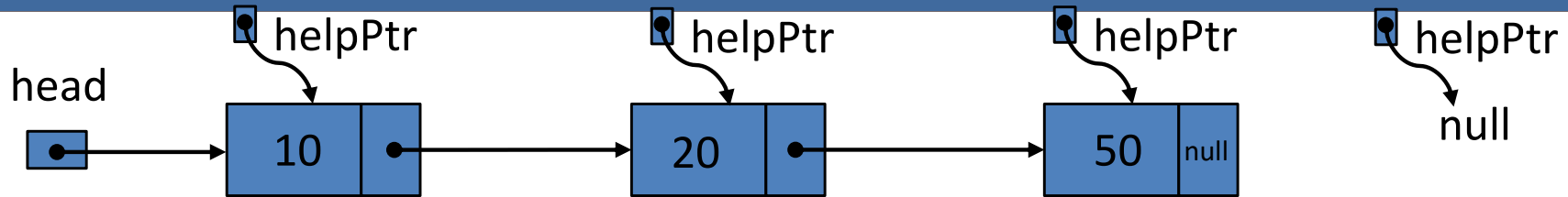
- Display the list data
- Head points to first node **10 20 50**

```
public void printAllNodes() {
    while (head != null) {
        System.out.print(head.data + " ");
        head = head.next;
    }
}
```

- The linked list is **no more accessible**
- Lost the address of first node

Linked List – Traverse

“No Bonus” marks for this course anymore



- Display the list data
- Head points to first node
- Use temporary object “helpPtr”

```

public void printAllNodes() {
    LLnode helpPtr = head;
    while (helpPtr != null) {
        System.out.print(helpPtr.data + " ");
        helpPtr = helpPtr.next;
    }
}
  
```

Linked List – Traverse

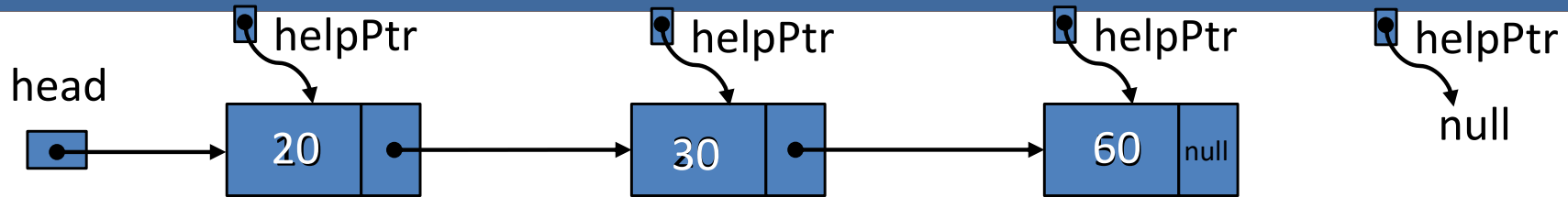
“No Bonus” marks for this course anymore

C
P
C
S

2

0

4

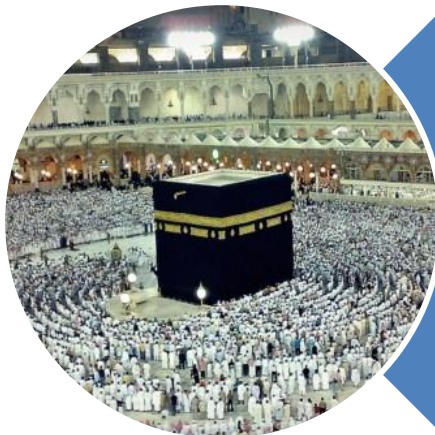


- Modify the content of each node
- Head points to first node
- Use temporary object “helpPtr”

```

public void printAllNodes() {
    LLnode helpPtr = head;
    while (helpPtr != null) {
        helpPtr.data = helpPtr.data + 10;
        helpPtr = helpPtr.next;
    }
}
  
```

Linked List – Modifying



Allah (Alone) is Sufficient for us,
and He is the Best Disposer of
affairs (for us). **(Quran 3 : 173)**